Capture the Flag 8: Enroll a New Student

Goal: Extend your multi-screen Student Directory by implementing student enrollment using form fields, controllers, validation, submission, and local data persistence while preserving your existing navigation, student details, and edit modal.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does.
- Understand syntax.
- Explain an error (not fix your error).
- Clarify documentation.

You may NOT ask AI to generate the solution/code for a flag.

🏴 Flag 0 — Create Your Feature Branch

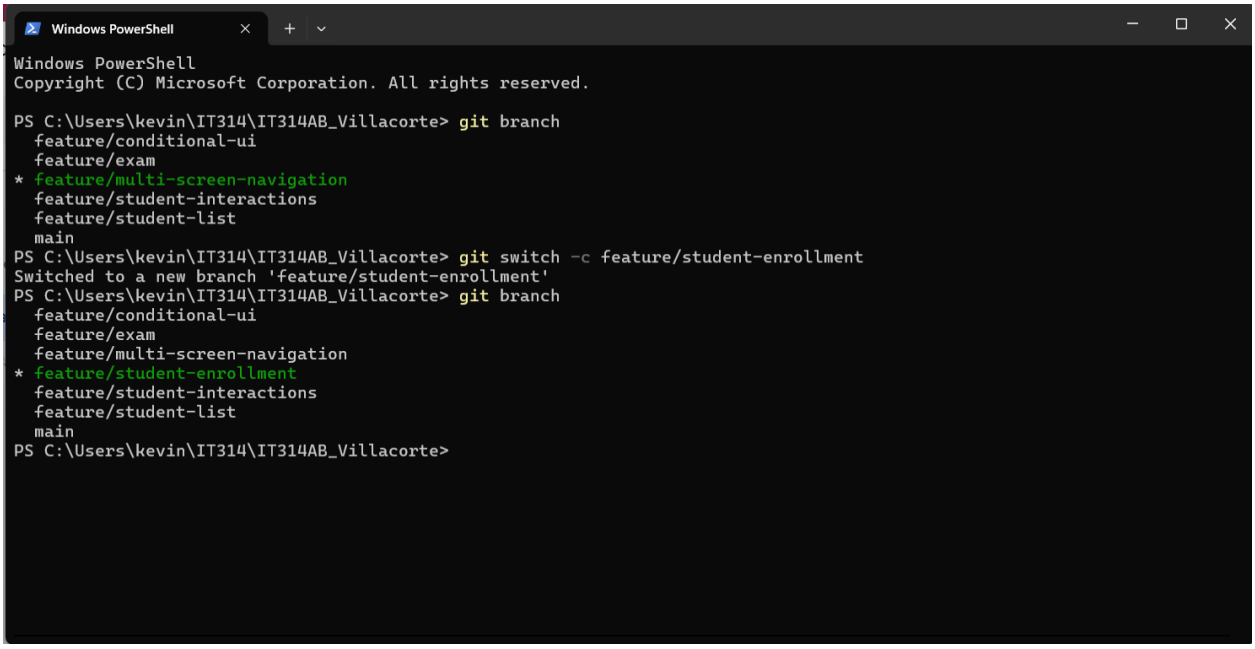
Objective: Create a new feature branch before implementing student enrollment.

Create:

`feature/student-enrollment`

Screenshot:

- Terminal showing your current branch.



🏴 Flag 1 — Build the Add Student Form

Objective: Transform your existing Add Student Screen into a functional enrollment form.

Add the following input fields:

- Student ID
- Full Name
- Course
- Year Level
- Email Address

Research how Flutter uses:

- `TextField` or `TextFormField`
- `TextEditingController`
- Form layouts and input fields

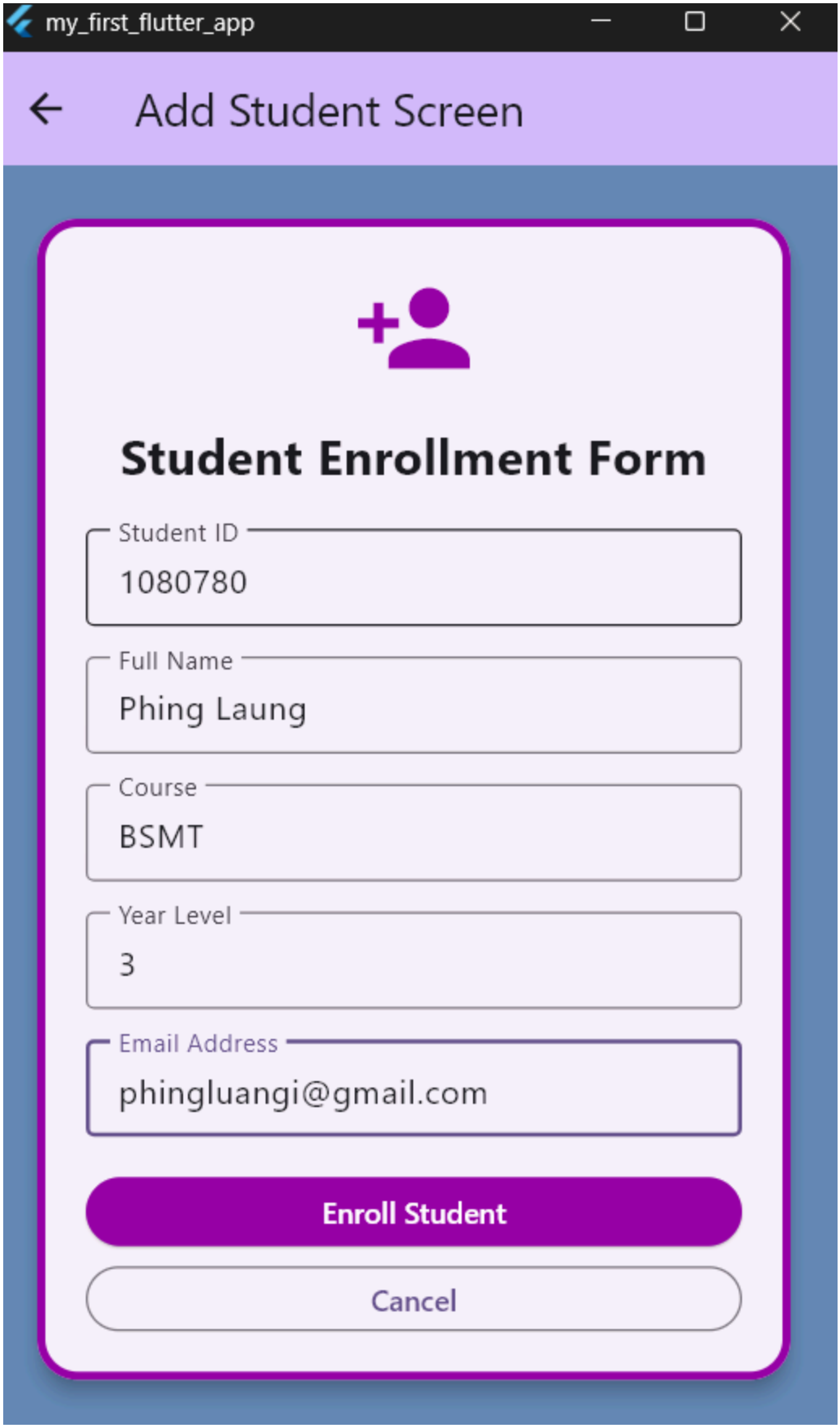
Test

The Add Student Screen should display all required fields and allow administrators to enter student information.

The screen must remain accessible through your existing Student List navigation.

Screenshot:

- Add Student Screen with all input fields.

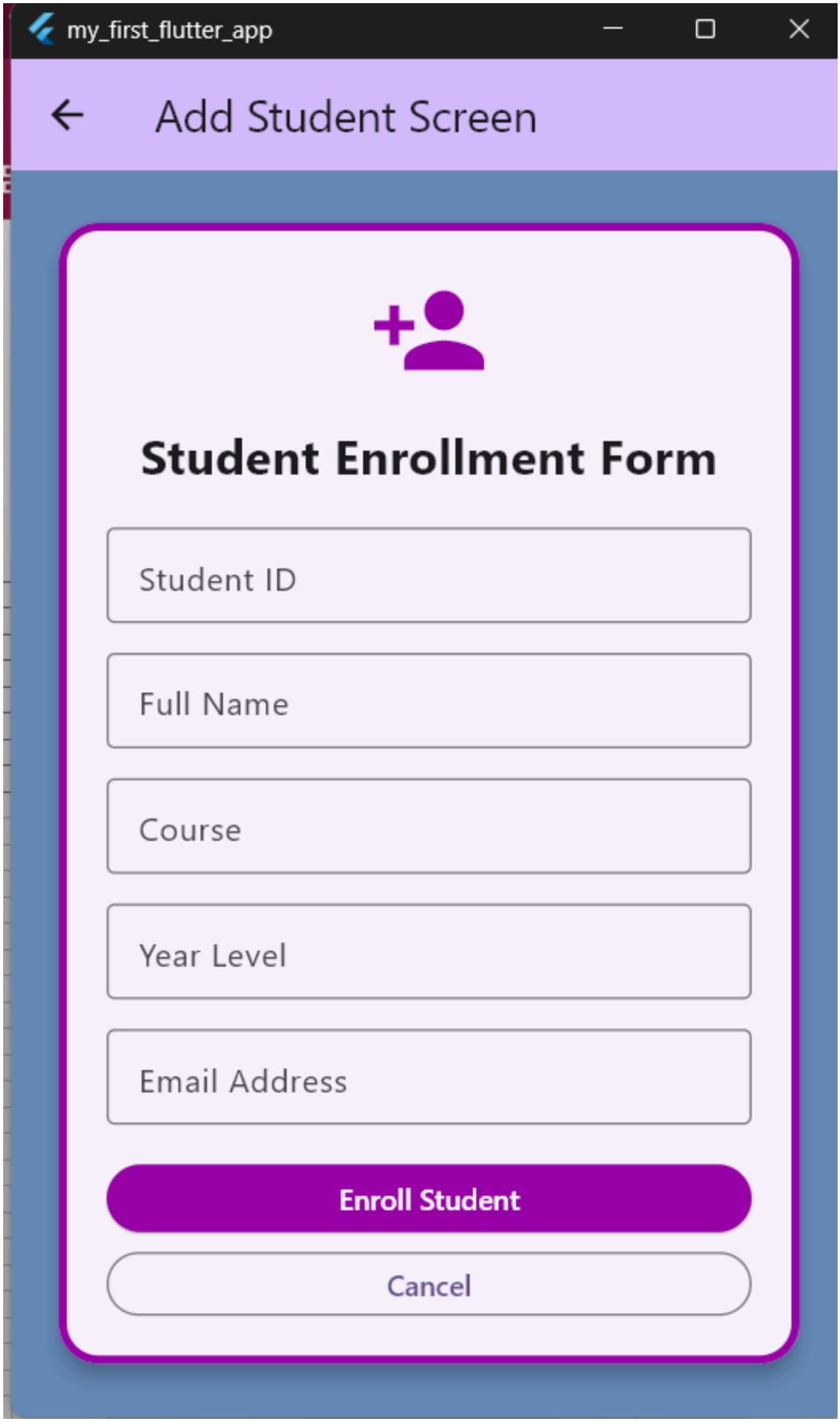

my_first_flutter_app
Add Student Screen
Student Enrollment Form
Student ID
1080780
Full Name
Phing Laung
Course
BSMT
Year Level
3
Email Address
phingluangi@gmail.com
Enroll Student
Cancel

- Controllers declared in your code.

```dart
class _AddStudentScreenState extends State<AddStudentScreen> {
  TextEditingController studentIdController = TextEditingController();
  TextEditingController nameController = TextEditingController();
  TextEditingController courseController = TextEditingController();
  TextEditingController yearController = TextEditingController();
  TextEditingController emailController = TextEditingController();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        title: const Text('Add Student Screen'),
      ), // AppBar
```

- Application displaying the enrollment form.


my_first_flutter_app
Add Student Screen
Student Enrollment Form
Student ID
Full Name
Course
Year Level
Email Address
Enroll Student
Cancel

🏴 Flag 2 — Validate Student Information

Objective: Prevent incomplete, invalid, or duplicate student information from being submitted.

Implement form validation with the following requirements:

- All required fields must be filled in.
- Student ID must not duplicate an existing student's ID.
- Email Address must follow a valid email format.
- Year Level must contain an appropriate year level value.

Research how Flutter handles form validation and displays validation messages.

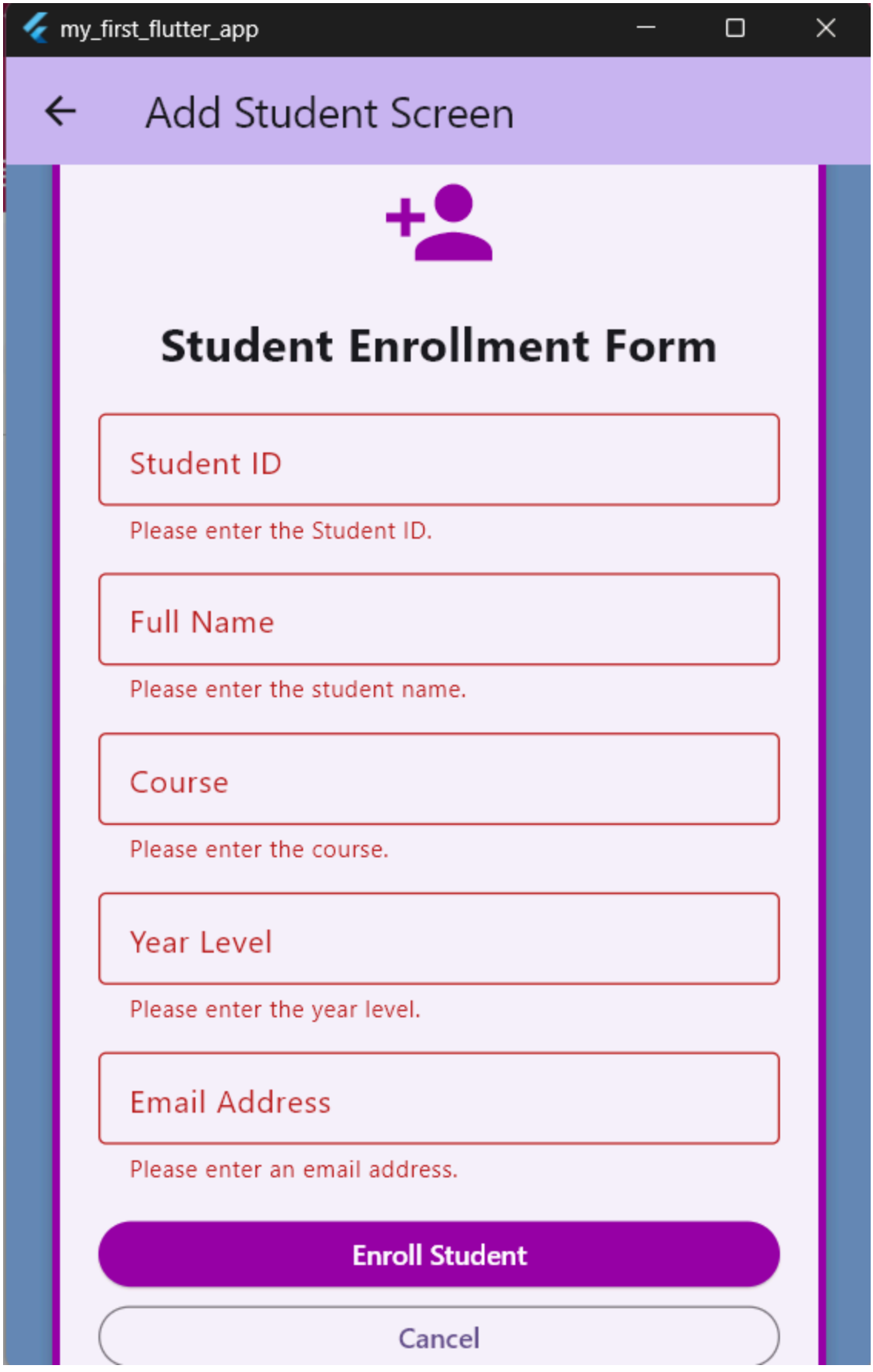
Test

1. Submit the form with all fields empty.
2. Enter an invalid email address.
3. Enter a Student ID that already exists.
4. Enter valid information into every field.

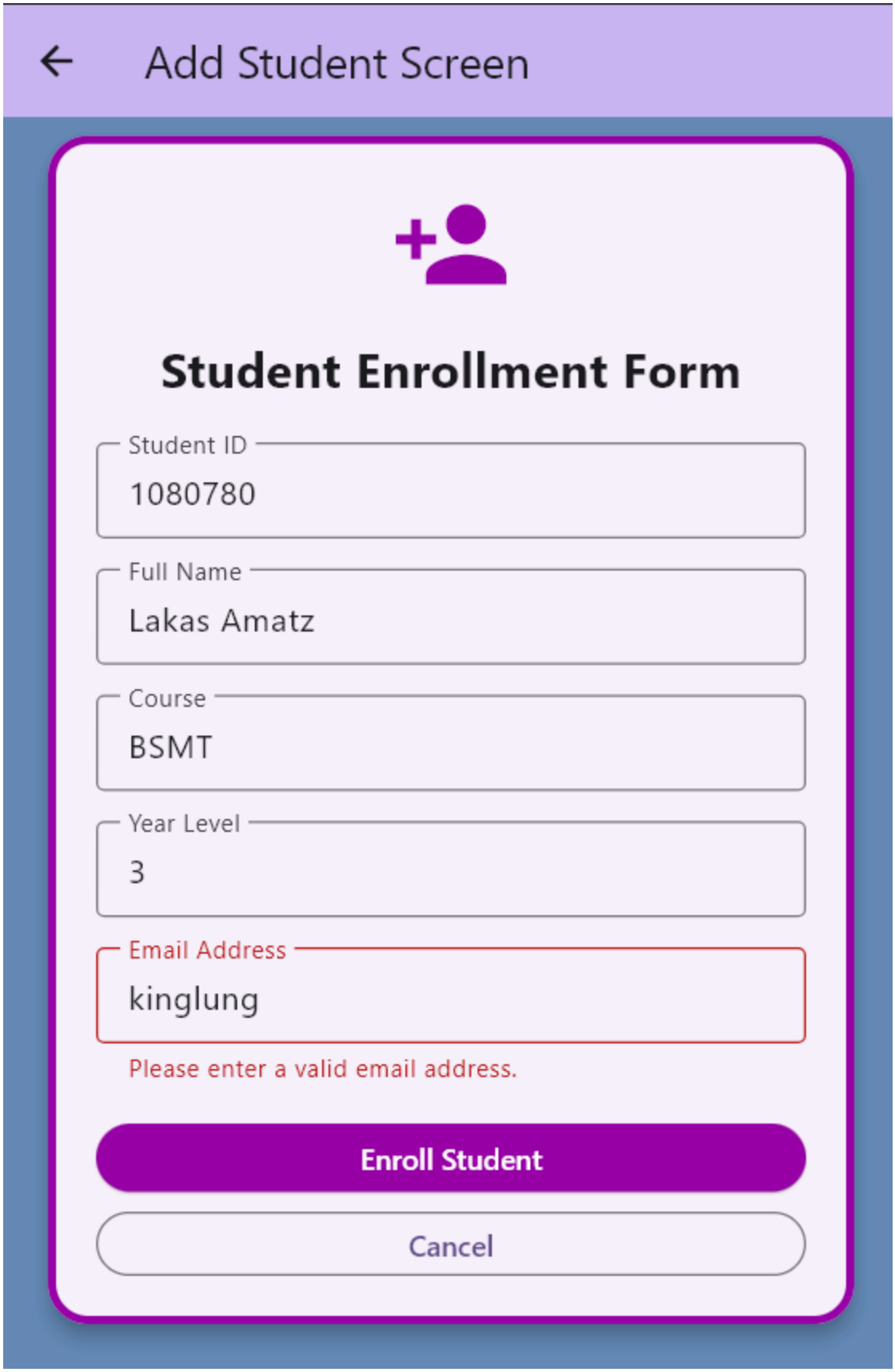
Invalid information should display appropriate validation messages and must not create a student record.

Screenshot:

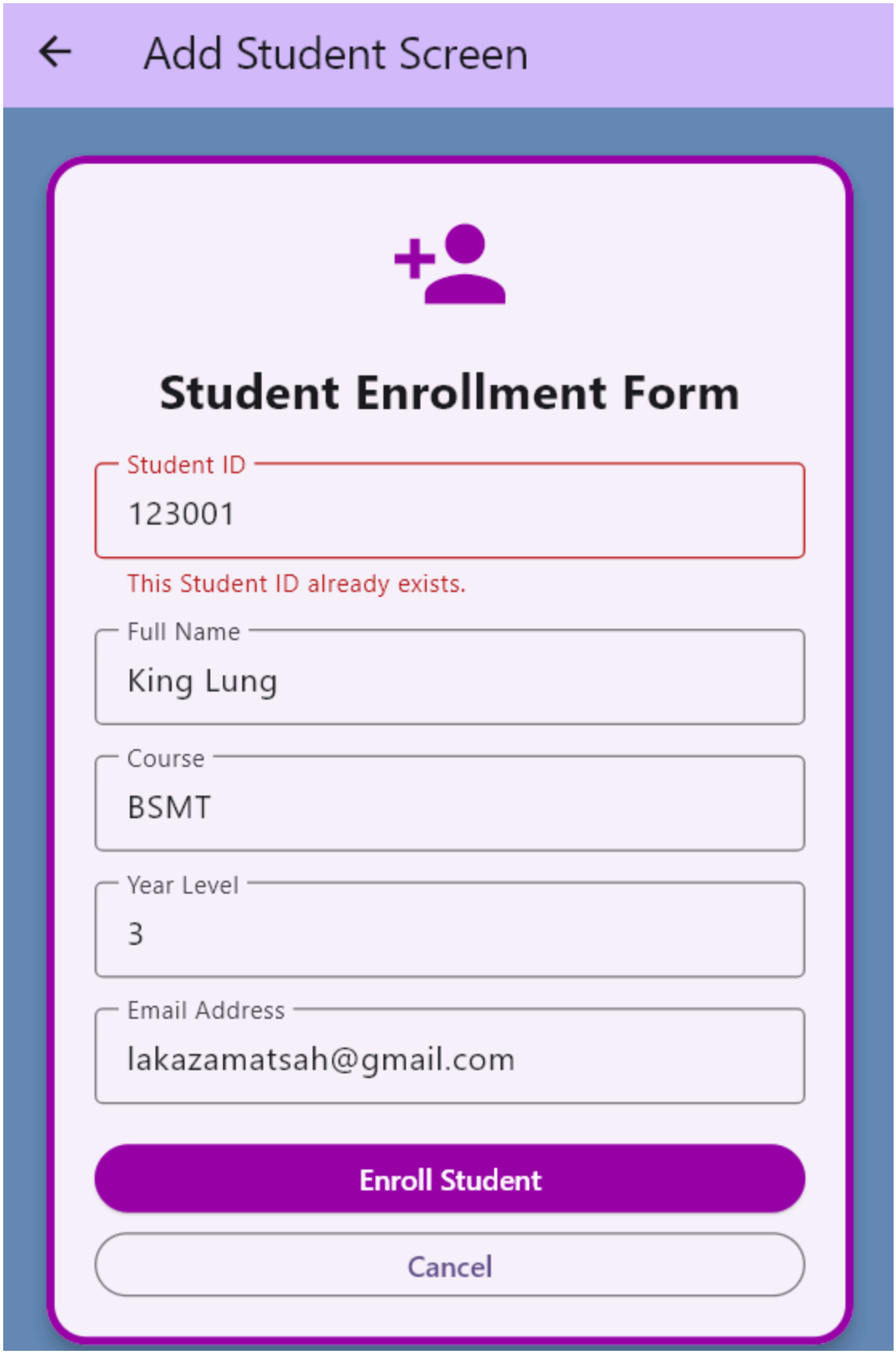
- Required-field validation.


my_first_flutter_app
Add Student Screen
Student Enrollment Form
Student ID
Please enter the Student ID.
Full Name
Please enter the student name.
Course
Please enter the course.
Year Level
Please enter the year level.
Email Address
Please enter an email address.
Enroll Student
Cancel

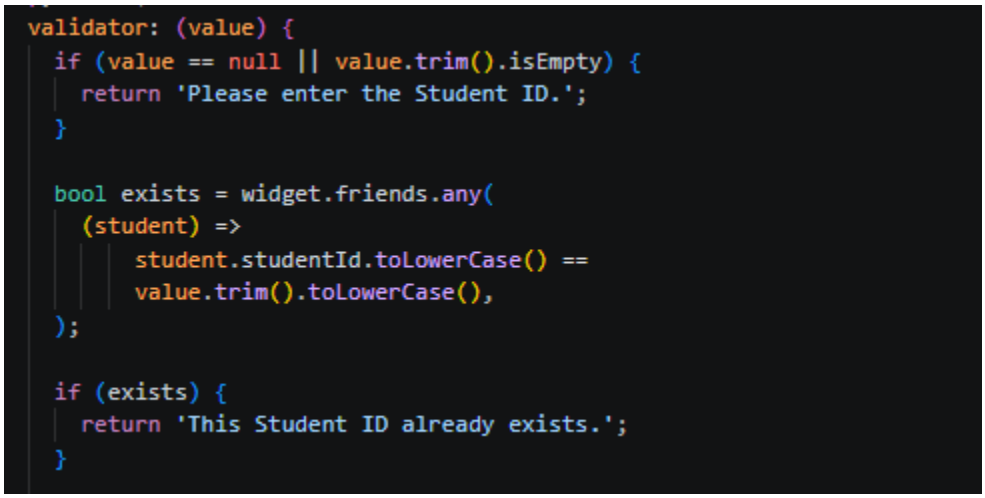
- Invalid email validation.


Add Student Screen
Student Enrollment Form
Student ID
1080780
Full Name
Lakas Amatz
Course
BSMT
Year Level
3
Email Address
kinglung
Please enter a valid email address.
Enroll Student
Cancel

- Duplicate Student ID validation.


Add Student Screen
Student Enrollment Form
Student ID
123001
This Student ID already exists.
Full Name
King Lung
Course
BSMT
Year Level
3
Email Address
lakazamatsah@gmail.com
Enroll Student
Cancel

- Validation logic in your code.



```
validator: (value) {
  if (value == null || value.trim().isEmpty) {
    return 'Please enter the Student ID.';
  }

  bool exists = widget.friends.any(
    (student) =>
        student.studentId.toLowerCase() ==
        value.trim().toLowerCase(),
  );

  if (exists) {
    return 'This Student ID already exists.';
  }
```

🏴 Flag 3 — Enroll and Display a New Student

Objective: Add a successfully enrolled student to your existing Student Directory.

When valid information is submitted:

- Create a new student record.
- Add the student to your existing student list.
- Return to the Student List Screen or update it appropriately.
- Display the newly enrolled student.
- Clear the form fields after successful submission.

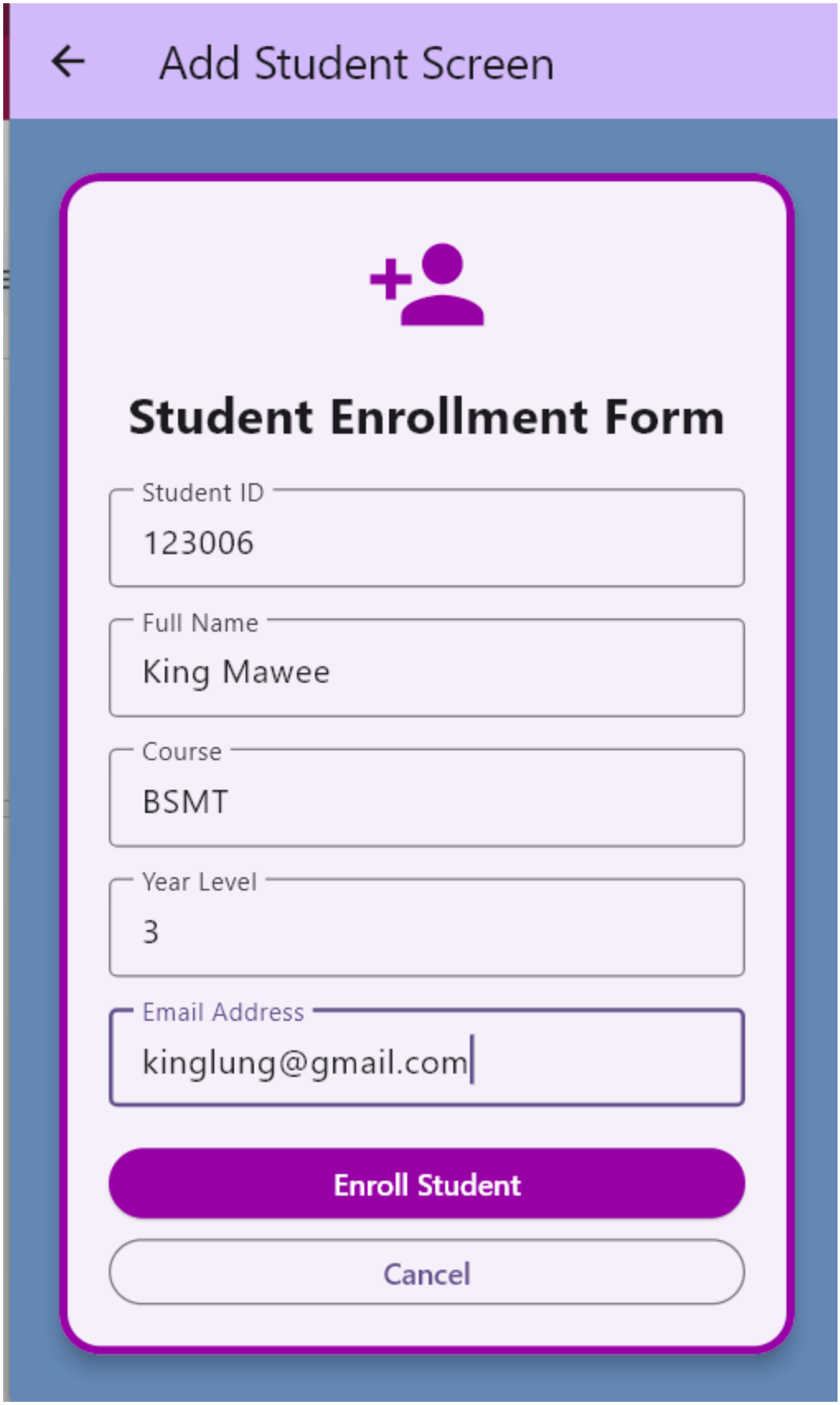
Research how Flutter handles form submission, updating application state, and returning from a screen using navigation.

Test

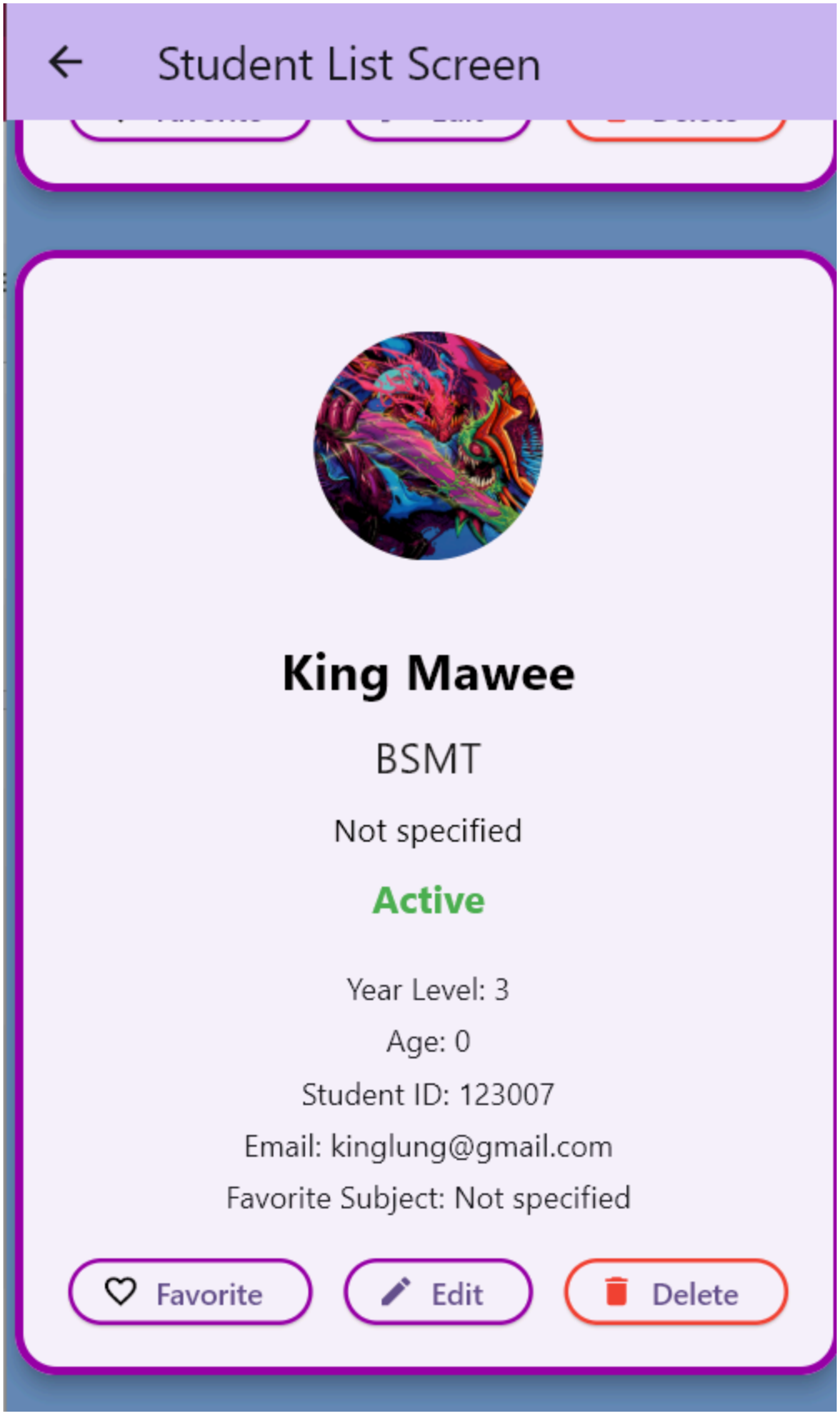
1. Open the Add Student Screen.
2. Enter valid student information.
3. Submit the form.
4. Verify that the new student appears in the Student List.
5. Select the new student.
6. Verify that the Student Details Screen displays the correct information.

Screenshot:

- Completed enrollment form.


Add Student Screen
Student Enrollment Form
Student ID
123006
Full Name
King Mawee
Course
BSMT
Year Level
3
Email Address
kinglung@gmail.com
Enroll Student
Cancel

- Newly enrolled student in the Student List.


Student List Screen
King Mawee
BSMT
Not specified
Active
Year Level: 3
Age: 0
Student ID: 123007
Email: kinglung@gmail.com
Favorite Subject: Not specified
Favorite
Edit
Delete

- Student Details Screen displaying the new student's information.

Student Details Screen



King Mawee

Student ID: 123007

Course: BSMT

Status: Active

Year Level: 3

Age: 0

Hobby: Not specified

Email: kinglung@gmail.com

Favorite Subject: Not specified

Edit

- Submission and navigation logic.

```dart
Friend? newStudent = await Navigator.pushNamed(
  context,
  '/add-student',
  arguments: friends,
) as Friend?;

if (newStudent != null) {
  setState(() {
    friends.add(newStudent);
  });
```

🏴 Flag 4 — Save Student Records Locally

Objective: Preserve student records even after the application is closed and restarted.

Research Flutter-compatible local data persistence options and choose an appropriate method.

Your implementation must:

- Save student records locally.
- Load saved records when the application starts.
- Preserve newly enrolled students after restarting the application.
- Handle situations where no saved records exist yet.

Test

1. Enroll a new student.
2. Close and restart the application.
3. Verify that the newly enrolled student still appears in the Student List.
4. Open the student's details and verify that the information remains correct.

Screenshot:

- Student List displaying saved records.

Student List Screen

Favorite Edit Delete



King Fritz

BSMT

Not specified

Active

Year Level: 3

Age: 0

Student ID: 123007

Email: kingfries@gmail.com

Favorite Subject: Not specified

Favorite Edit Delete

- Local persistence implementation.

```dart
Future<void> saveStudents(List<Friend> friends) async {
  final prefs = await SharedPreferences.getInstance();
  final studentsJson =
      jsonEncode(friends.map((friend) => friend.toJson()).toList());
  await prefs.setString('students', studentsJson);
```

- Student Details Screen displaying a saved student after restarting the application.

← Student Details Screen



King Fritz

Student ID: 123007

Course: BSMT

Status: Active

Year Level: 3

Age: 0

Hobby: Not specified

Email: kingfries@gmail.com

Favorite Subject: Not specified

Edit

🏴 Flag 5 — Keep Existing Features Functional

Objective: Ensure that enrollment works with the navigation and interactions implemented in CTF 7.

Your application must retain:

- Student List Screen.
- Student Details Screen.
- Student information passed between screens.
- Edit modal.
- Add Student Screen.
- Existing named routes.

The enrollment workflow must integrate with these existing features rather than replacing them with a single screen.

Test

1. Navigate from Student List to Add Student.
2. Enroll a new student.
3. Open the newly enrolled student's details.
4. Open the Edit modal.
5. Modify the student's information.
6. Close the modal and verify the updated information.
7. Navigate back to the Student List.
8. Verify that the student information remains consistent.

Screenshot:

- Student List with the newly enrolled student.

Student List Screen

Email: kingfries@gmail.com

Favorite Subject: Not specified

Favorite Edit Delete



Ezung Chisil

BSED

Badminton

Active

Year Level: 4

Age: 21

Student ID: 123008

Email: chisil@gmail.com

Favorite Subject: Math

Favorite Edit Delete

- Student Details displaying the correct information.

Student Details Screen



Ezung Chisil

Student ID: 123008

Course: BSED

Status: Active

Year Level: 4

Age: 21

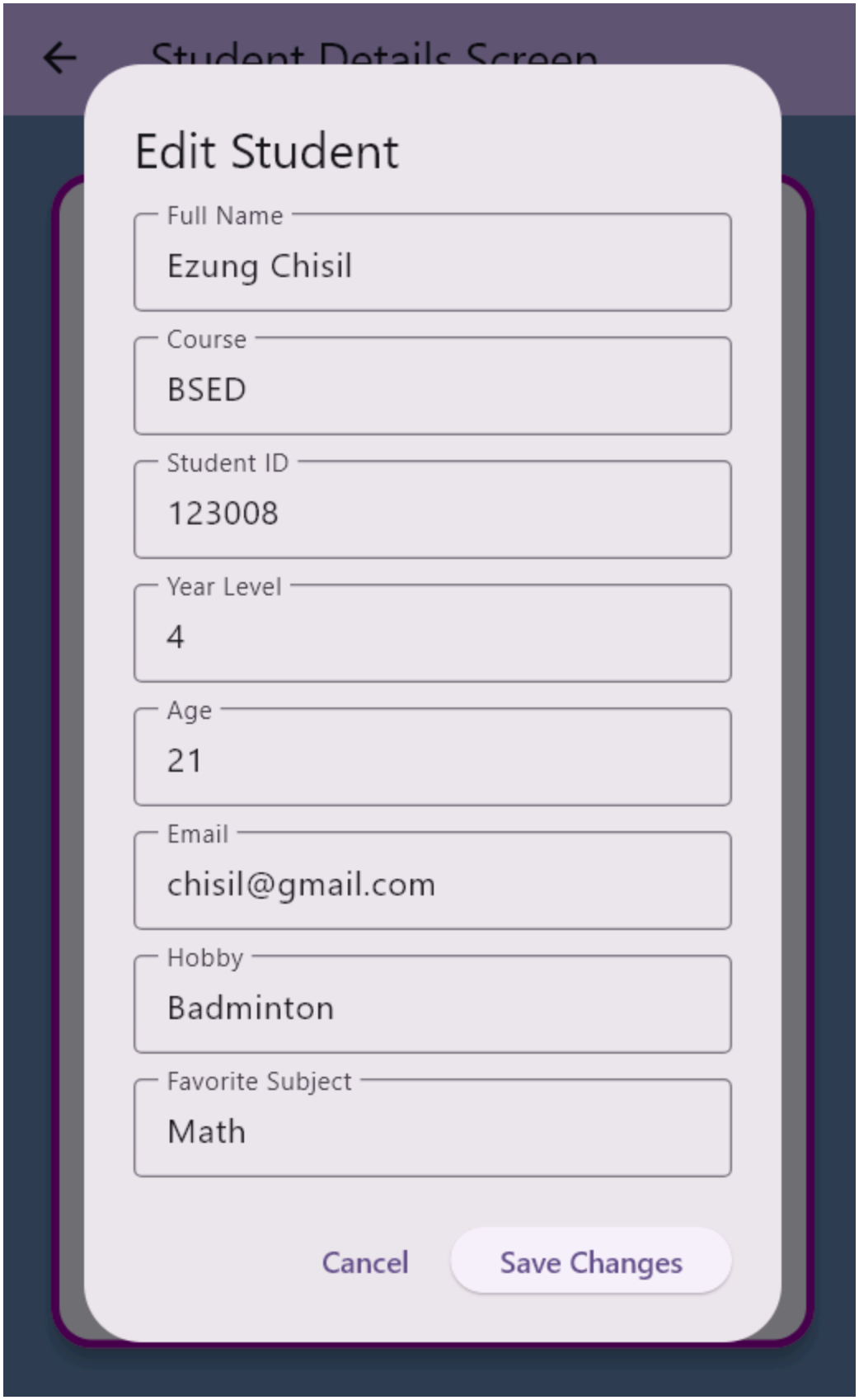
Hobby: Badminton

Email: chisil@gmail.com

Favorite Subject: Math

Edit Student

- Edit modal functioning correctly.


Student Details Screen
Edit Student
Full Name
Ezung Chisil
Course
BSED
Student ID
123008
Year Level
4
Age
21
Email
chisil@gmail.com
Hobby
Badminton
Favorite Subject
Math
Cancel
Save Changes

- Existing navigation working with the enrollment feature.

```
ElevatedButton.icon(
  onPressed: () {
    Navigator.pushNamed(context, '/student-list');
  },
  icon: const Icon(Icons.people),
  label: const Text('Open Student List'),
```

🏴 Flag 6 — Synchronize Changes with Local Data

Objective: Ensure that creating, editing, and deleting students updates the locally saved records.

Your application should keep its student list and locally persisted data consistent.

Verify that:

- Newly enrolled students are saved.
- Edited student information is saved.
- Deleted student records remain deleted after restarting.
- Student IDs remain unique.
- The Student List reflects the latest student records.

Research how to update and remove persisted records using your chosen local storage method.

Test

1. Enroll a new student.
2. Edit the student's information using the modal.
3. Delete a student using your existing delete functionality, if implemented.
4. Restart the application.
5. Verify that the saved records reflect the changes.

Screenshot:

- Student List after enrollment and editing.

Student List Screen

Email: kingfries@gmail.com

Favorite Subject: Not specified

Favorite Edit Delete



Ezung Chisil

BSED

Badminton

Active

Year Level: 4

Age: 21

Student ID: 123008

Email: chisil@gmail.com

Favorite Subject: Math

Favorite Edit Delete

- Updated student details.

Student Details Screen



Ezung Chisil

Student ID: 123008

Course: BSED

Status: Active

Year Level: 4

Age: 21

Hobby: Badminton

Email: chisil@gmail.com

Favorite Subject: Math

Edit Student

- Student records after restarting the application.

Student List Screen

Not specified

Active

Year Level: 3

Age: 0

Student ID: 123007

Email: kingfries@gmail.com

Favorite Subject: Not specified

Favorite Edit Delete



Ezung Chisil

BSED

Badminton

Active

Year Level: 4

Age: 21

- Persistence logic for creating, updating, and deleting records.

```dart
Future<void> addStudent() async {
  final newStudent = await Navigator.pushNamed(
    context,
    '/add-student',
    arguments: friends,
  ) as Friend?;
```

```dart
  Future<void> editStudent() async {
    final shouldSave = await showDialog<bool>
      context: context,
      builder: (context) {
        return EditStudentDialog(
          friend: widget.friend,
          friends: widget.friends,
        );
      },
    );
```

```dart
Future<void> deleteStudent(int index) async {
  setState(() {
    friends.removeAt(index);
  });

  await saveStudents(friends);
}
```

🏴 Flag 7 — Verify the Complete Enrollment Workflow

Objective: Demonstrate that the Student Directory supports enrollment, validation, navigation, editing, and local data persistence.

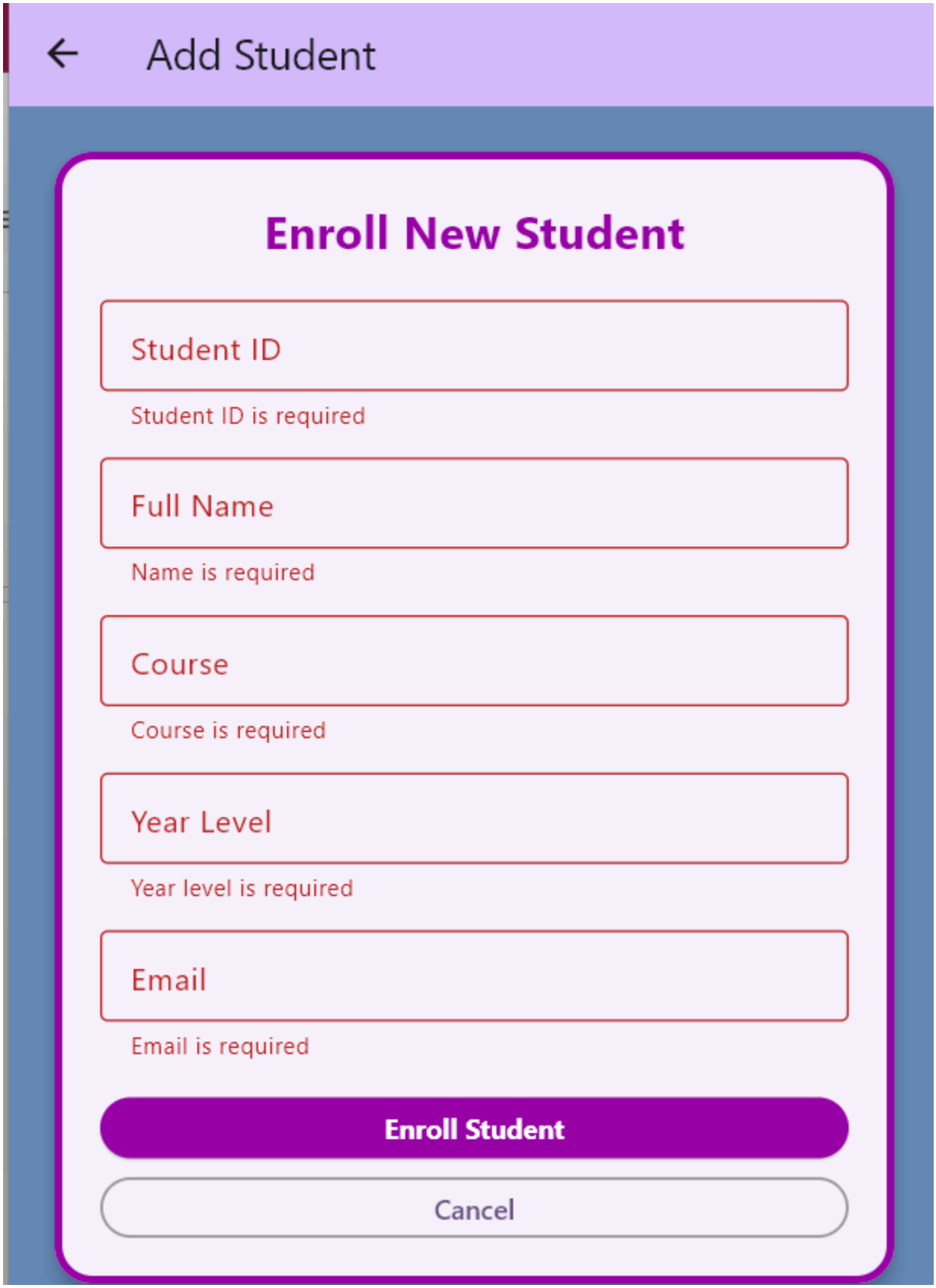
Test Scenario

1. Launch the application.
2. Verify that saved student records load.
3. Navigate to Add Student.
4. Submit an empty form.
5. Submit an invalid email address.
6. Attempt to enroll a student with a duplicate Student ID.
7. Enter valid student information.
8. Submit the form successfully.
9. Verify that the student appears in the Student List.
10. Open Student Details and verify the student's information.
11. Edit the student using the modal.
12. Restart the application.

13. Verify that the enrolled student and updated information remain available.

Screenshot:

- Enrollment form with validation errors.


Add Student
Enroll New Student
Student ID
Student ID is required
Full Name
Name is required
Course
Course is required
Year Level
Year level is required
Email
Email is required
Enroll Student
Cancel

- Successful enrollment.
- Student List displaying the new student.

Student List Screen

Student ID: 123008
Email: chisil@gmail.com
Favorite Subject: Math

Favorite Edit Delete



Cryptic Immortal

BSIT

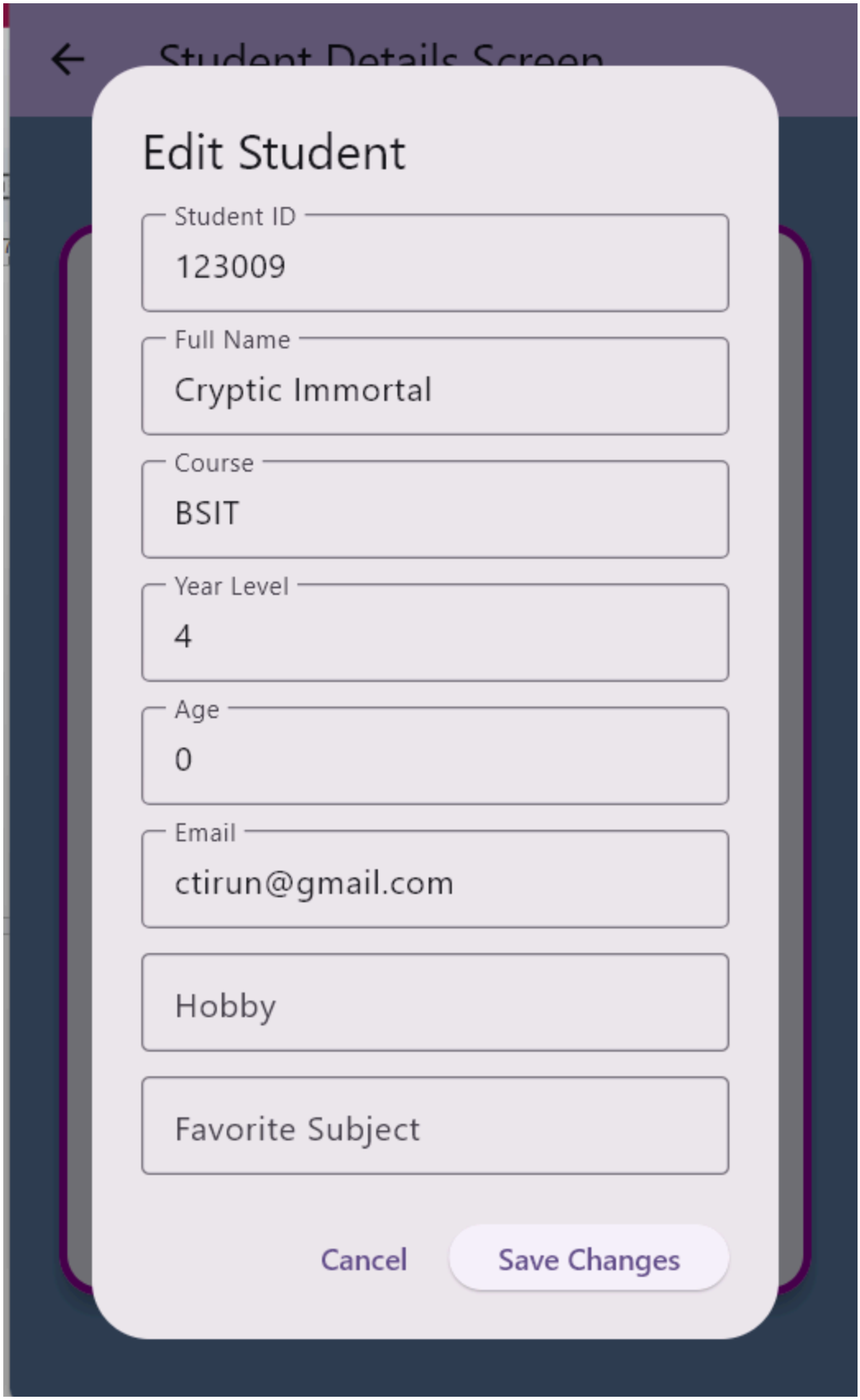
Active

Year Level: 4
Age: 0
Student ID: 123009
Email: ctirun@gmail.com
Favorite Subject:

Favorite Edit Delete

Student enrolled and saved successfully!

- Student Details and Edit modal.


Student Details Screen
Edit Student
Student ID
123009
Full Name
Cryptic Immortal
Course
BSIT
Year Level
4
Age
0
Email
ctirun@gmail.com
Hobby
Favorite Subject
Cancel
Save Changes

Student Details Screen



Cryptic Immortal

Student ID: 123009

Course: BSIT

Status: Active

Year Level: 4

Age: 0

Hobby:

Email: ctirun@gmail.com

Favorite Subject:

Edit Student

- Saved student information after restarting the application.

Student List Screen

Student ID: 123008
Email: chisil@gmail.com
Favorite Subject: Math

Favorite Edit Delete



Cryptic Immortal

BSIT

Active

Year Level: 4
Age: 0
Student ID: 123009
Email: ctirun@gmail.com
Favorite Subject:

Favorite Edit Delete

🏴 Flag 8 — Save Your Progress

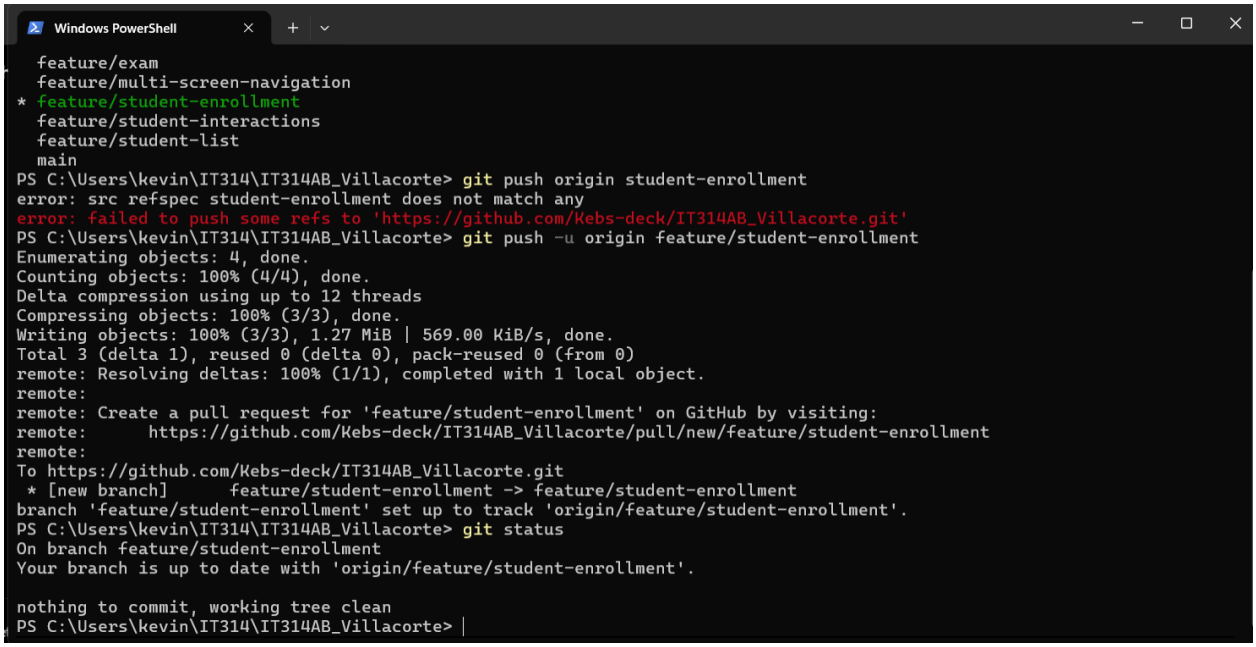
Objective: Commit and push your completed Student Enrollment feature.

Use a proper Git commit message.

Branch:

`feature/student-enrollment`

Screenshot:



- Successful commit.
- Current branch.
- Terminal showing the commit.
- Successful push to the remote repository.